ClawGym

РУКОВОДСТВО

Кейсы, решения и корзины

Как готовить задачи для обучения и проверки агента в ClawGym: идея простыми словами, устройство файлов, уроки, сценарии наставника и лучшие практики.

Идея и роли | Кейсы | Уроки | Решения | Корзины | Чек-листы

Как читать это руководство

ClawGym — тренажёр, на котором агент Pagurus учится вести обращения второй линии поддержки. Чтобы тренажёр работал, кто-то должен придумать задачи, описать, как помогает наставник, и собрать из этого занятия и экзамены. Руководство — для тех, кто этим занимается: методистов, аналитиков и разработчиков стенда.

Сначала объясняется идея: зачем нужны кейсы, решения, корзины и уроки и как они работают вместе. Затем — каждая сущность отдельно, с примерами из демо-справочников ClawGym и лучшими практиками. В конце — чек-листы.

Есть 10 минут
Часть 1: идея, роли и пять принципов.

Пишете задачи
Части 2 и 3: кейсы и уроки.

Ставите эксперименты
Части 4 и 5: решения и корзины.

Содержание

ЧАСТЬ 1 **Идея простыми словами**
Тренажёр и его роли, четыре кирпича, как агент учится и как это измерить, пять принципов, словарь

ЧАСТЬ 2 **Кейс — задача для агента**
Из чего состоит, тематика, факты, заглушки, эталон, оценка, лучшие практики, кейс целиком

ЧАСТЬ 3 **Уроки — правила, которым учит наставник**
Зачем нужны, путь урока до скилла, метрики поправок, как написать хороший урок

ЧАСТЬ 4 **Решения — роль наставника**
Что делает LLM-оператор, режимы, стили помощи, как писать инструкцию, как ставить эксперименты

ЧАСТЬ 5 **Корзины — занятие и экзамен**
Обучение и проверка, расписание /skill-curator, схема «до и после», заморозка проверки

ЧАСТЬ 6 **Шпаргалка**
Путь от идеи до прогона, чек-листы, частые сообщения проверки, где читать подробнее

Пути вида `cases/…`, `baskets/…` считаются от каталога справочников `config/`. Готовые примеры из руководства лежат в демонстрационном наборе `dev/config/`: он устроен так же. Подробный справочник полей — в репозитории: `docs/guide/` и `docs/FORMATS.md`.

ЧАСТЬ 1

Идея простыми словами

Агент учится как новый сотрудник — на поправках опытных коллег. ClawGym превращает это в повторяемые занятия и экзамены.

1.1 Тренажёр для агента

Агент Pagurus помогает сотрудникам второй линии поддержки банка. Он читает обращение клиента, смотрит данные в системах банка, предлагает действия — например, выставить экспертизу или закрыть обращение с ответом — и пишет ответ клиенту.

Учится агент так же, как новый сотрудник. Наставник видит ошибку, останавливает её и объясняет правило; после смены разобранные случаи становятся опытом. ClawGym — тренажёр для таких занятий: одно и то же обращение можно провести сколько угодно раз, без живых клиентов и без риска, а потом измерить, чему агент научился.

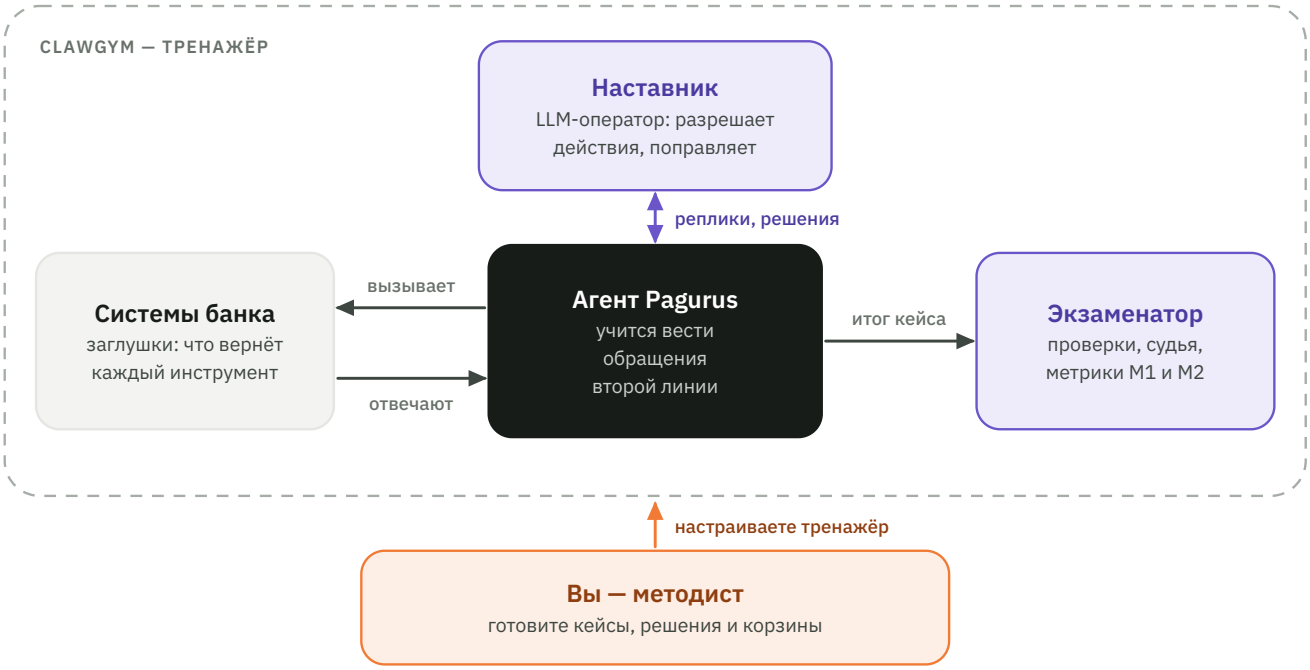


Рис. 1. Роли в тренажёре. Агент решает обращение. ClawGym играет системы банка, наставника и экзаменатора, а вы готовите для них материалы.

Роль	Кто играет	Что делает
Системы банка	Заглушки инструментов из кейса. ClawGym — MCP-сервер агента	Отвечают на вызовы агента так, как ответила бы настоящая система
Наставник	LLM-оператор — модель GigaChat в роли сотрудника второй линии	Разрешает или отклоняет действия, поправляет агента, объясняет правила, отвечает на вопросы
Экзаменатор	Автоматические проверки действий и LLM-судья	Решает, пройден ли кейс, и считает метрики обучения
Методист	Вы	Готовите задачи, сценарии наставника и программы занятий

Главная идея

Для агента ClawGym выглядит как настоящий оркестратор: те же запросы и тот же набор инструментов. Агент не знает правильного ответа и не видит, как его оценивают. Поэтому всё, что агент должен узнать о клиенте, вы кладёте в ответы инструментов, а правильное решение — в эталон. Эталон видят только наставник и экзаменатор.

1.2 Четыре кирпича

Всё, что вы готовите, — файлы в каталоге `config/`. Их четыре вида, и каждый отвечает на свой вопрос. Файлы ссылаются друг на друга по идентификаторам: настройка прогона — на корзину, корзина — на решения, решение — на кейс.

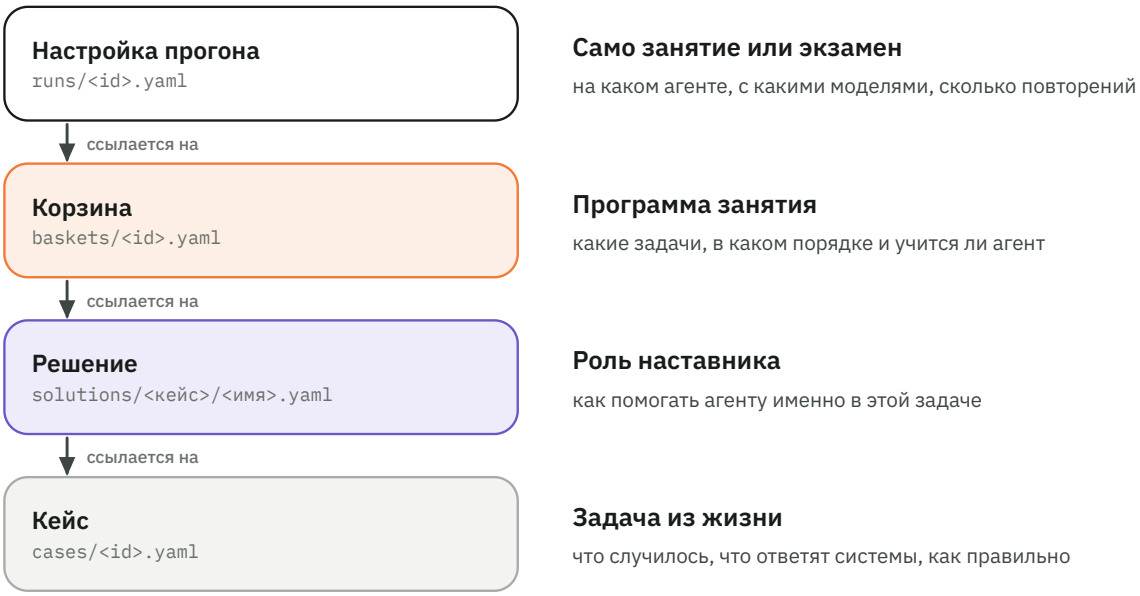


Рис. 2. Из чего собирается прогон. Чем ниже кирпич, тем чаще его переиспользуют: один кейс входит во многие решения, одно решение — во многие корзины.

Файл	Простыми словами	Пример из демо
Кейс	Задача из жизни: что написал клиент, что покажут системы, как правильно	«Сумма по карте заблокирована, но не списана»
Решение	Как наставник ведёт себя именно в этой задаче	«Объясняет сразу», «Только факты»
Корзина	Программа: какие решения, в каком порядке, занятие это или экзамен	«Обучение · СберСпасибо и операции по картам»
Настройка прогона	Конкретное занятие: на каком агенте, с какими моделями, сколько раз	`demo-train`, `demo-check`

Из этого устройства следуют три правила, на которых держится всё остальное:

- **Задача отдельно, наставник отдельно.** Один кейс проходят с разными наставниками, а данные и эталон у них общие.
- **Корзина собирается из решений, а не из кейсов.** Поэтому один кейс может встретиться в корзине несколько раз — с разными наставниками или с одним.
- **Занятие и экзамен не смешиваются.** Тип корзины — обучающая или проверяющая — решает, учится ли агент во время прогона.

1.3 Как агент учится

Агент учится на поправках. Когда он ошибается, наставник отклоняет действие и объясняет правило. Когда обращение закончено, ClawGym отправляет агенту команду `/case-capture`, и агент сохраняет разобранный случай вместе с поправкой. Уже в следующем обращении той же тематики агент видит поправку как свежий урок. Время от времени ClawGym отправляет `/skill-curator`: агент переносит накопленные уроки в свои выученные скиллы.

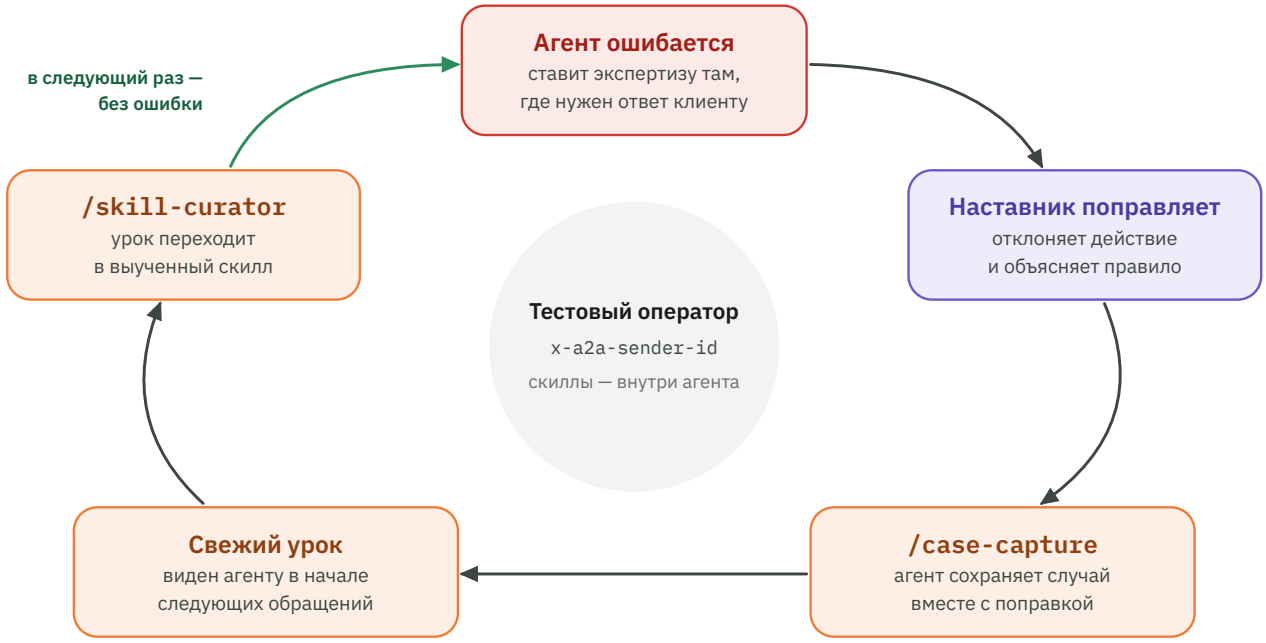


Рис. 3. Цикл обучения. Ошибка, поправка, сохранённый случай, свежий урок, выученный скилл — и следующее похожее обращение агент решает без подсказки.

Скиллы живут внутри агента, у тестового оператора. Тестовый оператор — это идентификатор `x-a2a-sender-id`, с которым ClawGym обращается к агенту. Новый оператор начинает со стартовых скиллов сборки агента. Прогон, продолжающий работу на том же операторе, получает всё, чему тот научился раньше.

Что делает ClawGym, а что — агент

ClawGym проводит занятие: подставляет ответы систем, играет наставника, отправляет служебные команды и считает метрики. Уроки извлекает и хранит сам агент. ClawGym не видит его скиллы и не правит их, а о том, что сделал `/skill-curator`, узнаёт по строке результата команды.

1.4 Как понять, что агент научился

Нужно сравнить два экзамена на одном наборе задач — до занятия и после. Так устроена схема аналитика из трёх прогонов:

1. **Проверка до.** Проверяющая корзина на новом тестовом операторе: что агент умеет со стартовыми скиллами.
2. **Обучение.** Обучающая корзина: наставник поправляет, агент сохраняет случаи и уроки.
3. **Проверка после.** Та же проверяющая корзина на операторе, который прошёл обучение.

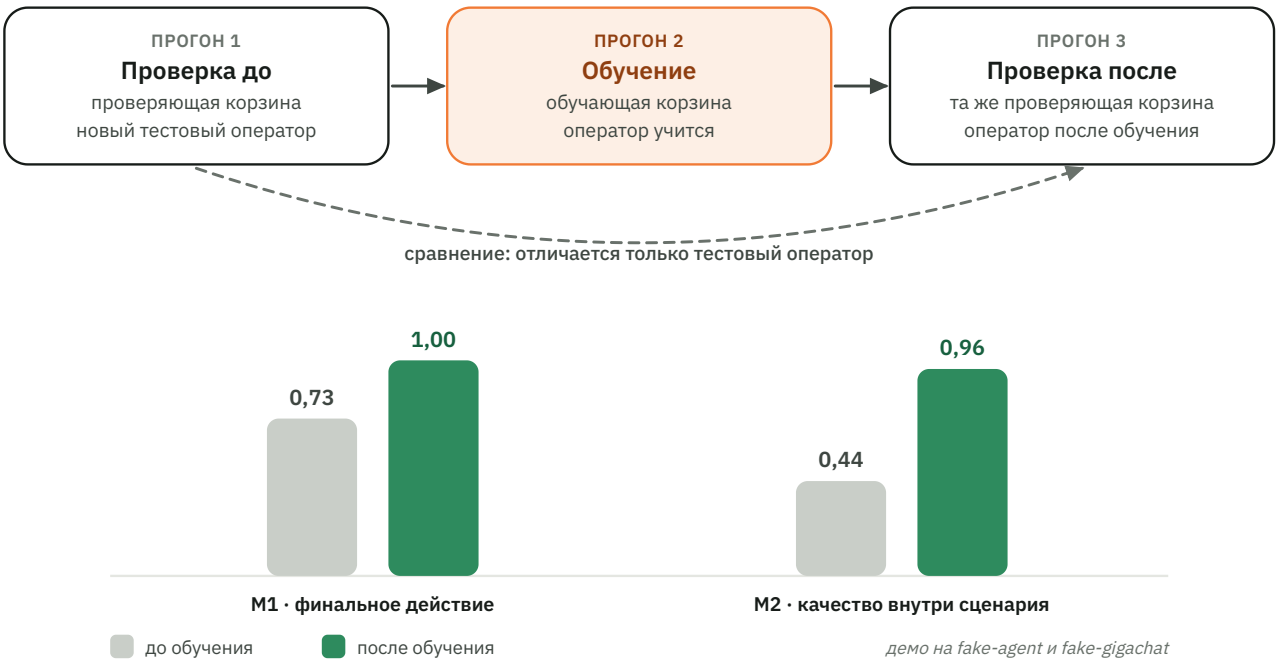


Рис. 4. Схема «до и после». Внизу — результат офлайн-демо: агент перестал путать блокировку суммы со спорной операцией.

Прогоны «до» и «после» сопоставимы, только если отличается один тестовый оператор: та же корзина, те же кейсы и решения, те же модели и настройки стенда. ClawGym сверяет это сам — по снимкам файлов — и в сравнении показывает, что различается.

Что измеряем	Из чего	На какой вопрос отвечает
Итог кейса	Проверки действий и критерии ответа	Решил ли агент задачу правильно?
Поправки	Сколько уроков пришлось объяснить	Много ли агенту подсказывали?
Повторные поправки	Урок, который уже объясняли в этой линии	Усвоил ли агент правило с первого раза?
M1	Класс финального действия: ответ и закрытие, экспертиза…	Верное ли итоговое решение принял агент?
M2	Точность набора инструментов и итогового текста	Насколько чисто агент дошёл до финала?

1.5 Пять принципов

1

Как на ПРОМе

Агент получает только то, что получил бы от оркестратора: стартовое сообщение, ответы инструментов и реплики. Правильный ответ ему не показывают.

2

Одна задача — одно умение

Кейс проверяет одно правило с одним правильным финалом. Тогда понятно, чему агент научился и где ошибся.

3

Данные отдельно от наставника

Кейс хранит факты и эталон, решение — поведение наставника. Их меняют независимо друг от друга.

4

Учим и проверяем раздельно

Обучающая корзина меняет скиллы оператора, проверяющая — нет. Экзамен всегда проходит без подсказок.

5

Честное сравнение

Между проверками «до» и «после» меняется только тестовый оператор. Проверяющую корзину, её кейсы и решения не трогают всю серию — иначе сравнивать нечего.

1.6 Короткий словарь

Термин	Значение
Кейс	Задача: обращение клиента, ответы систем и эталон правильного решения
Решение	Роль наставника в одном кейсе — инструкция LLM-оператору
Корзина	Упорядоченный список решений: обучающий или проверяющий
Прогон	Запуск корзины на агенте с выбранными моделями по настройке из `runs/`
Линия	Одно повторение корзины в прогоне, со своим тестовым оператором
Тестовый оператор	`x-a2a-sender-id`, у которого в агенте живут скиллы: стартовые от сборки и выученные
LLM-оператор, наставник	Модель, которая играет сотрудника второй линии: отвечает агенту и решает по подтверждениям
Судья, экзаменатор	Отдельная модель оценки: проверяет ответ по критериям и сравнивает итоговый текст с эталонным
Заглушка	Заранее заданный ответ инструмента на вызов агента
Эталон	Правильное решение кейса: процедура, ответ, критерии, действия, уроки. Агенту не показывается
Урок	Правило с идентификатором, которое объясняет наставник
Поправка	Реплика или отклонение действия, которые исправляют агента; помечается уроком
Свежий урок	Поправка из сохранённого случая: агент видит её в следующих обращениях, пока `/skill-curator` не перенёс её в скилл
Подтверждение	Пауза перед действием агента, пока наставник не разрешит или не отклонит его
Терминальный инструмент	Действие, после которого кейс закрыт, например `get_close_with_answer`

ЧАСТЬ 2

Кейс — задача для агента

Что написал клиент, что покажут системы банка и как правильно поступить. Один YAML-файл в `cases/`.

2.1 Из чего состоит кейс

У кейса четыре части. Карточка нужна людям и интерфейсу. Ситуация и факты — наставнику и экзаменатору. Заглушки — это то, что увидит агент. Эталон — правильное решение. Агент получает только ответы инструментов: из них же собирается стартовое сообщение, с которого начинается обращение.

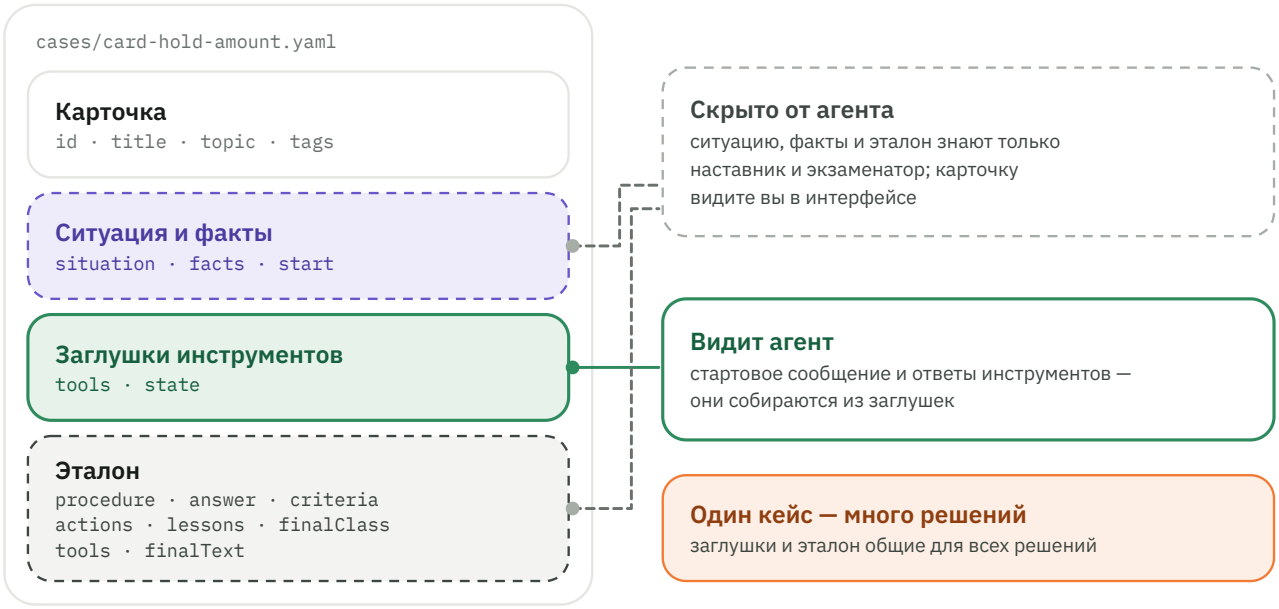


Рис. 5. Анатомия кейса на примере `card-hold-amount`. Зелёным отмечено то, что попадает к агенту, пунктиром — то, что от него скрыто: ситуацию и эталон читают наставник и экзаменатор.

2.2 Карточка и тематика

Поле	Что писать	Совет
`id`	Идентификатор: латиница, цифры, `. _ / -`. Совпадает с именем файла	Называйте по проблеме клиента: `card-hold-amount`, а не по ожидаемому решению
`title`	Название для людей	Глазами клиента: «Не начислены бонусы СберСпасибо»
`topic`	Тематика обращения	Точное название из SmartCare, одно для всех кейсов тематики
`tags`	Метки	Для поиска и группировки в интерфейсе

Почему тематика так важна

По тематике агент выбирает свои скиллы и свежие уроки: уроки той же тематики он видит первыми. Тематика приходит агенту дважды — в строке «Тематика:» стартового сообщения (её берут из заглушки `get_static_attributes_assigned`, поле `classifier.categoryName`) и в команде `/case-capture` (её берут из `topic`). Если строки расходятся, урок сохранится под одной тематикой, а искать его агент будет под другой.

2.3 Ситуация и факты

- **`situation`** — что произошло на самом деле и какое решение верное. Например: «Это не спорная операция: экспертиза не нужна, достаточно ответить клиенту и закрыть кейс». Читают наставник и экзаменатор; на экзамене наставник её не видит.
- **`facts`** — что наставник вправе рассказать агенту, если тот спросит: сумма, дата, продавец, сроки. На экзамене наставник знает о кейсе только факты: отвечает ими, а на остальное — «таких данных нет».

Факты — это данные, а не правила

На экзамене наставник может пересказать агенту любой факт. Если в фактах окажется «экспертизу не выставлять», наставник подскажет агенту решение, и экзамен потеряет смысл. Правила пишите в эталон: в процедуру и уроки. А данные — в факты: ситуацию на экзамене он не видит.

2.4 Заглушки: что ответят системы

Заглушка — заранее заданный ответ инструмента. Агент вызывает инструмент из контракта, а ClawGym отвечает тем, что написано в кейсе. Всё, что агент «увидит в системах банка», задаёт кейс.

```
tools:
  get_card_history:
    - id: hold-operation
      when: { id: CARD-TEST-3307 }          # ответ только для этой карты
      result:
        json: { operations: [{ date: "2026-10-25", sum: "7 800 ₽", status: HOLD }] }
    - id: other-card                        # другая карта — ошибка системы
      error: { message: Карта не найдена, code: CARD_NOT_FOUND }
```

- **Как выбирается ответ.** Подходят заглушки, у которых выполнены все условия `when`. Из них побеждает заглушка с большим числом условий, при равенстве — первая в файле. Нет подходящей в кейсе — ищется в общих заглушках `contracts/tool-defaults.yaml`. Нет и там — агент получает ошибку «данные не найдены».
- **Последовательность.** `sequence` — разные ответы на первый, второй и следующие вызовы: например, экспертиза создаётся, а повторная попытка получает «уже создана».
- **Состояние.** `state` и `effect` позволяют ответам меняться после действий агента: после `create_expertise` карточка обращения покажет, что экспертиза создана.
- **Ответ-ошибка — невыполненный вызов.** Он не засчитывается обязательному действию и не закрывает кейс.

Стартовое сообщение

Первая реплика агенту собирается по шаблону `contracts/start-message.txt` — так же, как у оркестратора: строка «Тематика:», `request_ref` и ответы четырёх инструментов: `get_cases_assigned`,

`get_static_attributes_assigned`, `get_client_universal_info`, `get_dynamic_attributes`. Ответ берётся как для вызова без аргументов, поэтому у этих заглушек не должно быть условий `when`, иначе строка выпадет. Проверить результат можно на странице кейса: «Стартовое сообщение — как его получит агент».

Заглушки неверного пути

Дайте ответы и тем инструментам, которые агент вызовет, если ошибётся. В кейсе о блокировке суммы есть заглушки для экспертизы: на экзамене ошибочная экспертиза выполнится, и M1 увидит неверный финал; на занятии наставник её отклонит и объяснит правило. Без заглушки ошибка агента превратится в сбой «данные не найдены».

Кавычки в YAML

Значение, которое начинается с `*`, YAML читает как ссылку на якорь: маску карты пишите в кавычках — `"****2208"`. Телефоны тоже в кавычках — `"+79990001122"`, иначе плюс пропадёт. Номера с ведущими нулями — `"0012"`.

2.5 Эталон: что считается правильным

Эталон — правильное решение задачи. Агенту он не передаётся. Каждая часть эталона нужна своему участнику: наставнику — чтобы вести диалог, экзаменатору — чтобы вынести итог, аналитику — чтобы посчитать метрики.

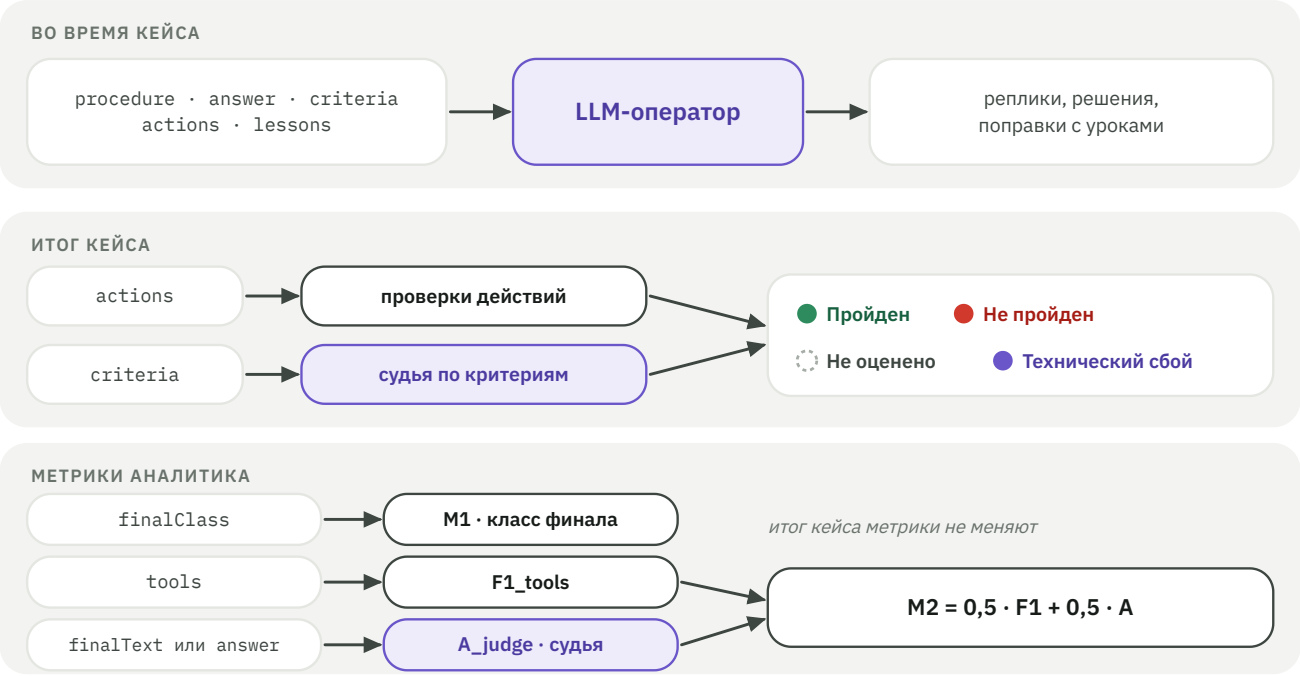


Рис. 6. Какая часть эталона питает какую оценку. Итог кейса и метрики аналитика M1, M2 считаются независимо.

Поле	Зачем	Совет
`procedure`	Шаги правильного решения. По ним наставник судит действия агента	Повелительно, с явными «не делать» и пометками, где нужно подтверждение
`answer`	Правильный ответ клиенту	Одно-два предложения
`criteria`	Утверждения о хорошем ответе для LLM-судьи	2–4 атомарных критерия, плюс «Не обещает того, чего нет в данных кейса»

`actions`	Обязательные и запрещённые действия, итоговое действие	Только существенное; запрет — для типичной ошибки
`lessons`	Правила, которым учит наставник	Подробно — в части 3
`finalClass`, `tools`, `finalText`	Эталон для M1 и M2: класс финала, минимальный набор инструментов, эталонный итоговый текст	Задавайте всегда, `finalClass` — явно

2.6 Как кейс оценивается

Сначала — автоматические проверки действий по журналу инструментов, затем LLM-судья по критериям.

- Не пройдена хотя бы одна проверка действий — кейс **не пройден**, что бы ни сказал судья.
- Проверки пройдены и судья подтвердил все критерии — кейс **пройден**. Нет критериев — кейс оценивается по действиям.
- Действия верны, но судьи нет или он дал сбой — **не оценено**, а не провал.
- Технический сбой — агента, оператора, `/reset` — кейс не оценивается вовсе.

Тонкости, о которых легко забыть

- Обязательное действие засчитывается, только если вызов **выполнен**: разрешён и получил ответ без ошибки.
- Запрещённое действие нарушает проверку при **любой попытке** — даже если наставник её отклонил.
- Наставник не видит условия на аргументы у действий: если они важны, опишите их словами в поле `text`.

2.7 Лучшие практики кейса

Делайте так

- **Одно умение и один правильный финал.** Две независимые ошибки в одном кейсе не дают понять, чему агент научился.
- **Только синтетические данные** — `CASE-TEST-…`, `CARD-TEST-…`: файлы кейсов попадают в снимок каждого прогона.
- **Одна история везде.** Ситуация, факты, заглушки, процедура и ответ говорят об одних суммах, датах и сроках.
- **Решающий факт — в ответе инструмента**, например статус `HOLD`, а не в догадке агента.
- **Заглушки стартового сообщения — без условий**, тематика в карточке и в заглушке совпадает.
- **Заглушки неверного пути** для типичной ошибки.
- **Эталон для метрик всегда:** `finalClass`, `tools`, а если финал не ответ клиенту — `finalText`.
- **Пройдите кейс вручную** до того, как класть его в корзину: «Кейсы и решения» → «Пройти вручную».

Частая ошибка	Что будет
Разные строки в `topic` и `categoryName`	Агент получит две тематики одного обращения, уроки разойдутся
Условие `when` у заглушки стартового сообщения	Строка выпадет из стартового сообщения

Правило в `facts`	На экзамене наставник подскажет решение
Финал `postpone_expertises` без `finalClass`	Кейс молча выпадет из M1
В эталоне `tools` — всё, что вызывал оператор	F1 штрафует агента за верный короткий путь

2.8 Пример: кейс целиком

Так выглядит демо-кейс `spasibo-bonus-missing`. Часть заглушек и критериев пропущена — на их месте `…`; полный файл — в `dev/config/cases/`.

```yaml
id: spasibo-bonus-missing
title: Не начислены бонусы СберСпасибо
topic: СберСпасибо / Начисление бонусов            # = categoryName в заглушке ниже
situation: >                                       # правда о кейсе — для наставника и судьи
  Клиент 12 октября купил товар у партнёра СберСпасибо и не получил бонусы. Покупка ждёт
  подтверждения партнёром, бонусы начислят после него. Признаков мошенничества нет.
facts:                                             # что наставник может сообщить по вопросу
  - Покупка 12 октября у партнёра «Тестовый партнёр» на 4 990 ₽.
  - Партнёр подтверждает покупки в течение 30 дней.
state: { case: open }
tools:                                             # … и ещё get_dynamic_attributes, get_spasibo_general_info
  get_cases_assigned:                              # стартовое сообщение: заглушка без when
    - id: case
      result:
        json: { caseId: CASE-TEST-1001, taskId: "{{execution.taskId}}", status: IN_WORK, clientMessage: "…" }
  get_static_attributes_assigned:
    - id: static
      result:
        json: { classifier: { categoryName: СберСпасибо / Начисление бонусов }, channel: CHAT }
  get_spasibo_history_info:                        # решающий факт — статус покупки
    - id: history
      when: { dateFrom: "2026-10-01" }
      result:
        json: { operations: [{ date: "2026-10-12", bonus: 249, status: PENDING_PARTNER_CONFIRMATION }] }
  get_close_with_answer:                           # финал: терминальный инструмент
    - id: closed
      delayMs: 600
      result: { json: { caseId: CASE-TEST-1001, status: CLOSED } }
      effect: { case: closed }
reference:
  procedure:
    - Запросить основную информацию Спасибо по клиенту
    - Проверить историю операций СберСпасибо за период покупки
    - "Ответить клиенту: бонусы ожидают подтверждения покупки партнёром"
    - Закрыть кейс с ответом после подтверждения оператора
  answer: Бонусы за покупку 12 октября ожидают подтверждения партнёром и будут начислены после него.
  finalClass: close_answer                         # M1
  tools: [get_spasibo_general_info, get_spasibo_history_info, get_close_with_answer]   # M2
  criteria:                                        # … и ещё bonus, no-promises
    - id: pending
      text: Сообщает, что бонусы ожидают подтверждения покупки партнёром
      lesson: L-spasibo-005
  actions:
    required:
      - tool: get_spasibo_history_info
        before: get_close_with_answer
        lesson: L-spasibo-004
      - tool: get_close_with_answer
    forbidden:
      - tool: create_expertise                     # типичная ошибка
        lesson: L-spasibo-006
    final: get_close_with_answer
  lessons:                                         # … и ещё L-spasibo-005, L-spasibo-006
    - id: L-spasibo-004
      rule: Перед ответом проверить историю операций СберСпасибо за период покупки.
```

ЧАСТЬ 3

Уроки — правила, которым учит наставник

Урок делает поправки одинаковыми и позволяет измерить главное: усвоил ли агент правило или его приходится объяснять снова.

3.1 Что такое урок

Представьте наставника, который ведёт тетрадь правил. У каждого правила есть номер и формулировка. Поправляя ученика, наставник отмечает в журнале: «объяснил правило № 1». Если через пару задач правило № 1 приходится объяснять снова, ученик его не усвоил.

Урок в ClawGym — такая запись тетради: короткий идентификатор и текст правила. Уроки лежат в эталоне кейса:

```
reference:
  lessons:
    - id: L-hold-001
      rule: "Блокировка суммы до подтверждения продавцом — не спорная операция:
        экспертизу не выставлять. Ответить клиенту: «Сумма заблокирована до подтверждения
        операции продавцом: после подтверждения она спишется, а без него вернётся на карту
        в течение 30 дней». Закрыть кейс с ответом."
```

Поле	Что это
`id`	Короткий стабильный идентификатор, например `L-hold-001`: тема и номер
`rule`	Самодостаточная формулировка правила: что делать, чего не делать, а если нужно — точный текст ответа

3.2 Зачем нужны уроки

1

Единая формулировка

Наставник опирается на текст урока, поэтому одно правило объясняют одинаково во всех кейсах и прогонах.

2

Измерение обучения

Каждая поправка помечается уроком. По пометкам ClawGym считает поправки и повторные поправки.

3

Связь с проверками

У критериев и действий есть поле `lesson`: нарушил агент проверку — наставник знает, какое правило напомнить.

3.3 Путь урока: от каталога до скилла

Каталог уроков видит только LLM-оператор. Когда агент ошибается, оператор поправляет его — отклоняет действие или отвечает репликой — и помечает поправку: «это поправка, урок `L-hold-001`». Дальше поправка идёт двумя дорогами.

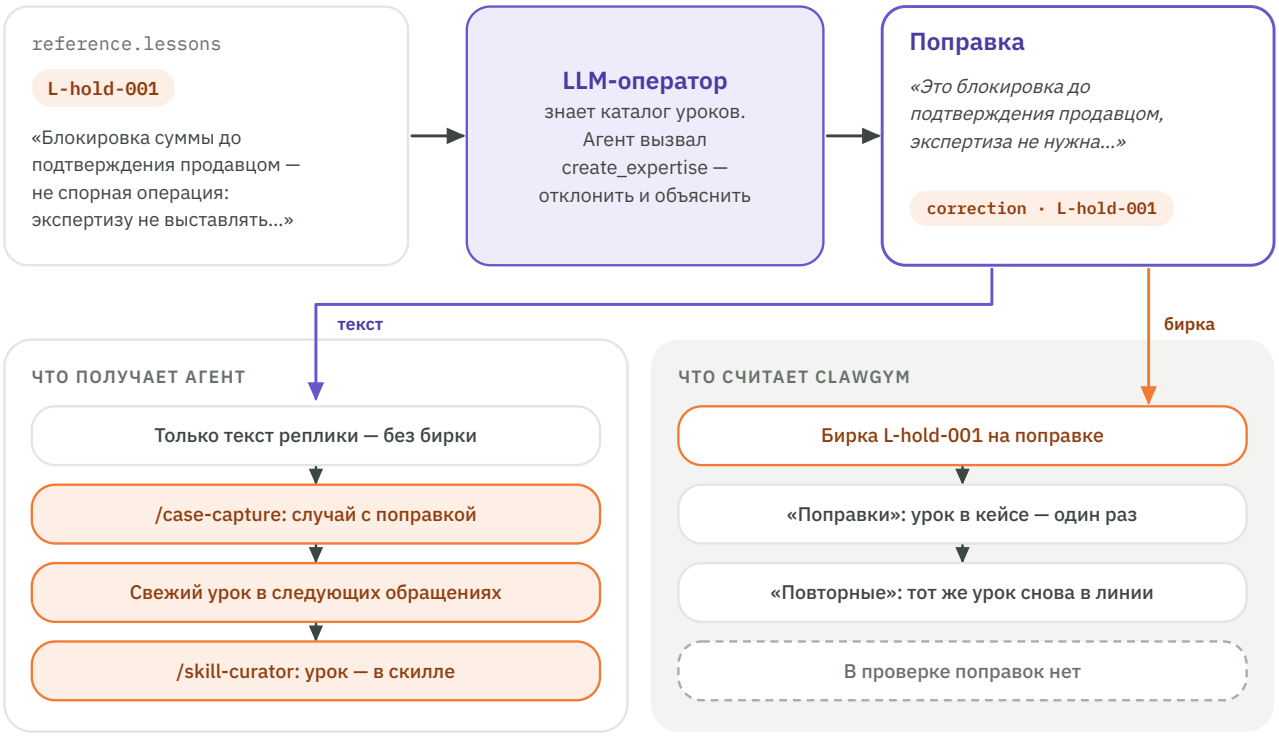


Рис. 7. Путь урока. Агент получает только текст поправки — без идентификатора. Бирку с уроком видит лишь ClawGym и считает по ней метрики.

- **Агент получает только текст реплики.** Ни идентификатора урока, ни пометок он не видит. Поэтому формулировка поправки важна: именно на ней агент и учится.
- **Дальше обучение идёт внутри агента.** `/case-capture` сохраняет случай с поправкой — у таких случаев сильный сигнал знания. В следующих обращениях той же тематики поправка видна агенту как свежий урок. `/skill-curator` переносит её в выученный скилл, после этого свежий урок исчезает: правило уже в скилле.
- **ClawGym считает по биркам.** Сколько уроков объяснили в кейсе, какие пришлось повторить, — всё это по пометкам оператора.

Как это выглядит в офлайн-демо

Тестовая модель `fake-gigachat` в стиле «Объясняет сразу» отправляет агенту текст `rule` дословно, а `fake-agent` ищет в нём знакомые фразы. Поэтому правила демо-кейсов сформулированы так, как их понимает `fake-agent`. Настоящий LLM-оператор формулирует поправку сам, опираясь на текст урока.

3.4 Где живут уроки

Где	Что значит
`reference.lessons` в кейсе	Основной каталог уроков кейса
`operator.lessons` в решении	Дополнительные уроки для одного сценария. Урок с тем же `id` заменяет урок кейса в этом решении
`lesson` у критерия	Какое правило нарушено, если критерий не выполнен
`lesson` у обязательного или запрещённого действия	Какое правило напомнить, если агент нарушил эту проверку

Проверка файлов предупреждает, если `lesson` у действия ссылается на урок, которого нет в каталоге. Ссылки у критериев не проверяются — следите за ними сами.

3.5 Как уроки превращаются в метрики

- **Поправка** — реплика или отклонение действия, которые исправляют агента или объясняют правило. Её помечает оператор.
- **Один урок в кейсе считается один раз**, даже если его объясняли трижды.
- **Поправка без урока считается каждая**: сравнить её не с чем.
- **Неизвестный урок не выдумывается.** Если оператор назвал урок не из каталога, пометка уходит в заметку и известным уроком не считается.
- **Повторная поправка** — урок, который уже объясняли раньше в этой линии обучения.
- **На экзамене поправок нет:** режим проверки их запрещает.

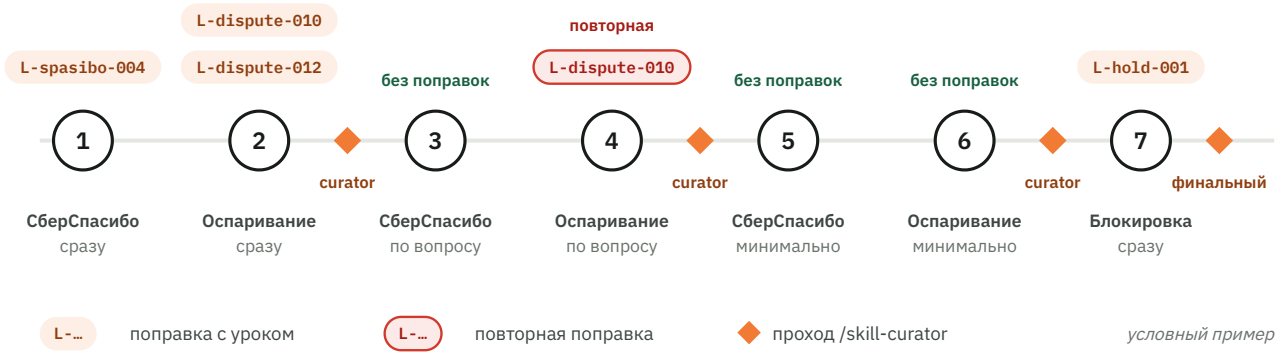


Рис. 8. Условный пример на демо-корзине обучения. Правило `L-dispute-010` объяснили во втором кейсе, а в четвёртом пришлось повторить — метрика «Повторные поправки» это покажет. Кейсы 3, 5 и 6 прошли без поправок.

Как читать результат

Хорошее обучение выглядит так: поправки по уроку есть в первых кейсах тематики, а в следующих их нет. Повторные поправки — сигнал, что агент правило не удержал. Причина может быть в формулировке поправки, в стиле наставника или в редких проходах `/skill-curator`.

3.6 Как написать хороший урок

Формула текста: **когда** правило действует, **что делать**, **чего не делать** и, если важна точная формулировка, — **текст ответа**.

Хорошо	Плохо	Почему
«Перед экспертизой посмотреть историю операций по карте за период операции.»	«Проверить всё как следует»	Одно проверяемое действие, а не пожелание
«Блокировка до подтверждения продавцом — не спорная операция: экспертизу не выставлять.»	«В кейсе с отелем на 7 800 ₽ экспертиза не нужна»	Правило переносится на другие обращения, а не привязано к одной сумме
«В ответе назвать срок рассмотрения — до 30 дней.»	«Ответить правильно»	Понятно, что именно добавить в ответ

Одно решение со следствием: «не экспертиза, а ответ и закрытие»	Три независимых требования в одном уроке	По повторной поправке видно, какое именно правило не усвоено

3.7 Лучшие практики уроков

Делайте так

- **1–3 урока на кейс** — только на то, в чём агент реально ошибается.
- **Одно правило — один `id` во всех кейсах.** Тогда повторная поправка в другом кейсе той же линии покажет, что правило не усвоено.
- **Привязывайте урок к проверке:** `lesson` у запрещённого или обязательного действия и у критерия.
- **Не меняйте `id` и смысл правила** внутри серии «до → обучение → после».
- **Урок решения** (`operator.lessons`) — только если правило нужно одному сценарию.
- **Урок — не факт.** Сумма, дата и статус — в `facts` и заглушках, правило — в уроке.
- **Формат идентификатора** — `L-<тема>-<номер>` , как в демо: `L-spasibo-004` , `L-hold-001` .

3.8 Уроки в демо-справочниках

Урок	Правило	С чем связан
`L-spasibo-004`	Перед ответом проверить историю операций СберСпасибо за период покупки	Обязательное `get_spasibo_history_info` до закрытия
`L-spasibo-005`	Сообщить, что бонусы ожидают подтверждения покупки партнёром	Критерий `pending`
`L-spasibo-006`	Экспертизу по бонусам, которые ждут подтверждения партнёра, не выставлять	Запрещённое `create_expertise`
`L-dispute-010`	Перед экспертизой посмотреть историю операций по карте	Обязательное `get_card_history` до экспертизы
`L-dispute-011`	В ответе назвать срок рассмотрения — до 30 дней	Критерий `deadline`
`L-dispute-012`	Спорную операцию не закрывать ответом — сначала экспертиза	Запрещённое `get_close_with_answer`
`L-hold-001`	Блокировка до подтверждения продавцом — не спорная операция: ответить и закрыть	Запрещённое `create_expertise` , критерий `release`

ЧАСТЬ 4

Решения — роль наставника

Решение описывает, как наставник ведёт себя в конкретной задаче: насколько помогает, когда объясняет правило и что отвечает на вопросы.

4.1 Что такое решение

Одно и то же правило можно преподать по-разному: сразу объяснить, дать агенту самому догадаться или только указать на ошибку. Решение описывает такую стратегию для одного кейса. Технически это небольшой файл: ссылка на кейс, цель эксперимента и инструкция LLM-оператору.

```yaml
# solutions/card-hold-amount/explain-now.yaml
case: card-hold-amount
title: «Объясняет сразу»
goal: перестанет ли агент выставлять экспертизу по блокировке суммы, если правило объяснили сразу.
operator:
  style: explain-now
  instruction: >
    Отклоняй действия, которые расходятся с эталоном, и сразу объясняй правило.
    Если ответ клиенту неполный, скажи, что добавить, и объясни почему.
```

4.2 Зачем отделять решение от кейса

1

Данные — один раз

Заглушки и эталон живут в кейсе. Решений у кейса может быть сколько угодно, и никто не копирует данные.

2

Эксперимент над обучением

Тот же кейс с разными наставниками показывает, какая помощь учит лучше.

3

Экзамен на тех же задачах

Для проверки берут тот же кейс с решением «Только факты»: наставник не подсказывает.

4.3 Что делает наставник во время кейса

LLM-оператора вызывают в двух случаях:

- **Агент хочет выполнить действие с подтверждением** — закрыть обращение, выставить экспертизу, сменить канал. Оператор разрешает или отклоняет вызов и может добавить реплику. Сама она агенту не уходит: до конца хода агент мог сделать нужное сам.
- **Агент закончил ход.** Оператор решает, что ответить, одобряет ли результат и не пора ли закончить кейс. Свои реплики к подтверждениям этого хода он видит как неотправленные и пишет агенту только то, что ещё нужно.

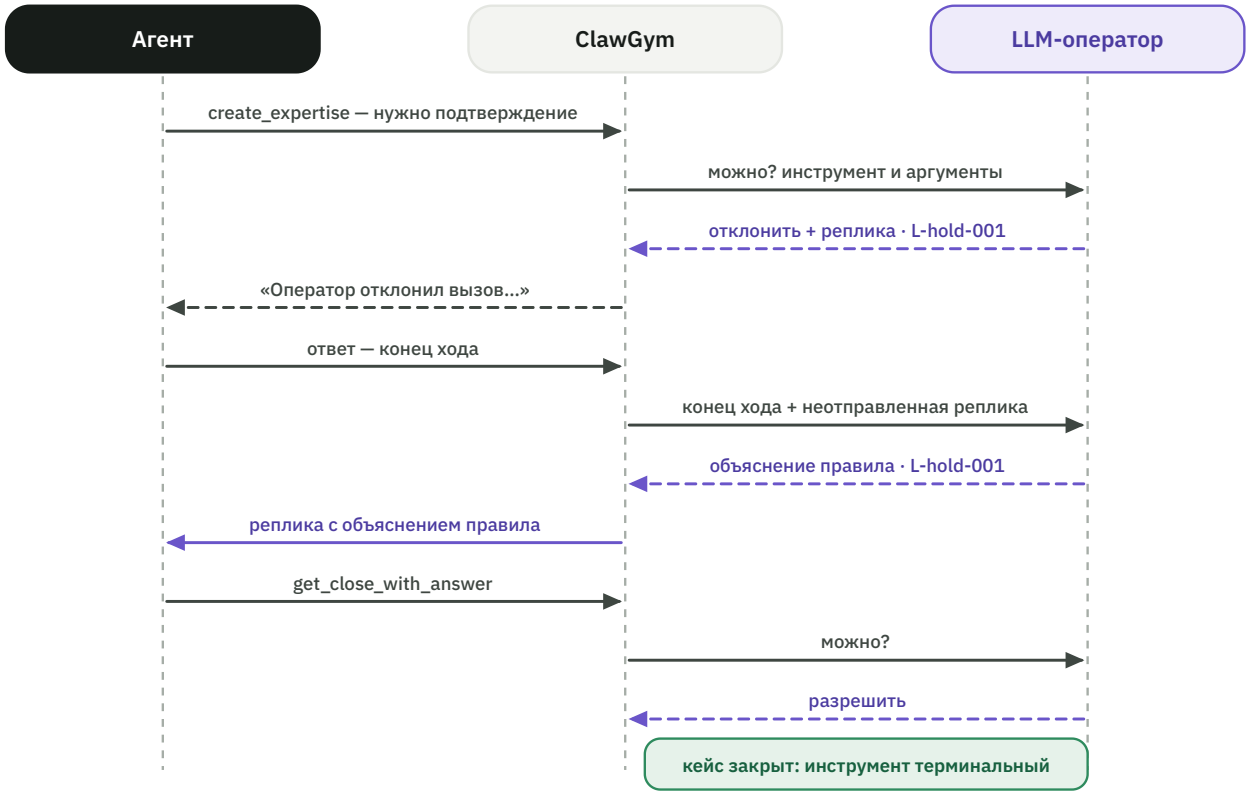


Рис. 9. Как проходит поправка. Оператор отклоняет экспертизу и помечает урок, агент получает отказ. После хода оператор видит свою неотправленную реплику и пишет агенту объяснение правила. Закрытие с ответом оператор разрешает, и кейс заканчивается: `get_close_with_answer` — терминальный инструмент.

Каждый раз оператор получает всё заново: режим корзины, ситуацию и факты, эталон, каталог уроков, инструкцию решения и журнал диалога. Памяти между вызовами нет, поэтому разные кейсы не смешиваются. Исключение — конец хода на экзамене: там оператор получает только факты и журнал. Общие правила поведения уже заданы подсказкой оператора: не выдумывать факты, не пересказывать эталон и не сообщать оценку, разрешать терминальное действие, только если финал верен, помечать поправки уроками. Инструкция решения задаёт только стратегию помощи.

4.4 Режим корзины важнее инструкции

	Обучающая корзина	Проверяющая корзина
Инструкция	Исполняется	В конце хода оператору не передаётся
Конец хода агента	Реплика по инструкции	Ответ о факте, напоминание или завершение
Поправки	Помечаются уроками, считаются в метриках	В конце хода выразить нечем. При подтверждении: попытка — переспрос, повтор — технический сбой
Подтверждения	Решает оператор	По умолчанию разрешаются автоматически, оператор не вызывается
Вопросы агента	По инструкции	Только факты кейса; нет факта — «таких данных нет»

Решение «Только факты» для экзамена всё равно заводят: его название и цель честно говорят в результатах, что это проверка, а не сценарий обучения. Что агент получает на экзамене — в разделе 5.1.

4.5 Стили: лестница помощи

В демо четыре стиля. Они выстраиваются в лестницу — от большей помощи к меньшей. На верхних ступенях агент учится, на нижней сдаёт экзамен.

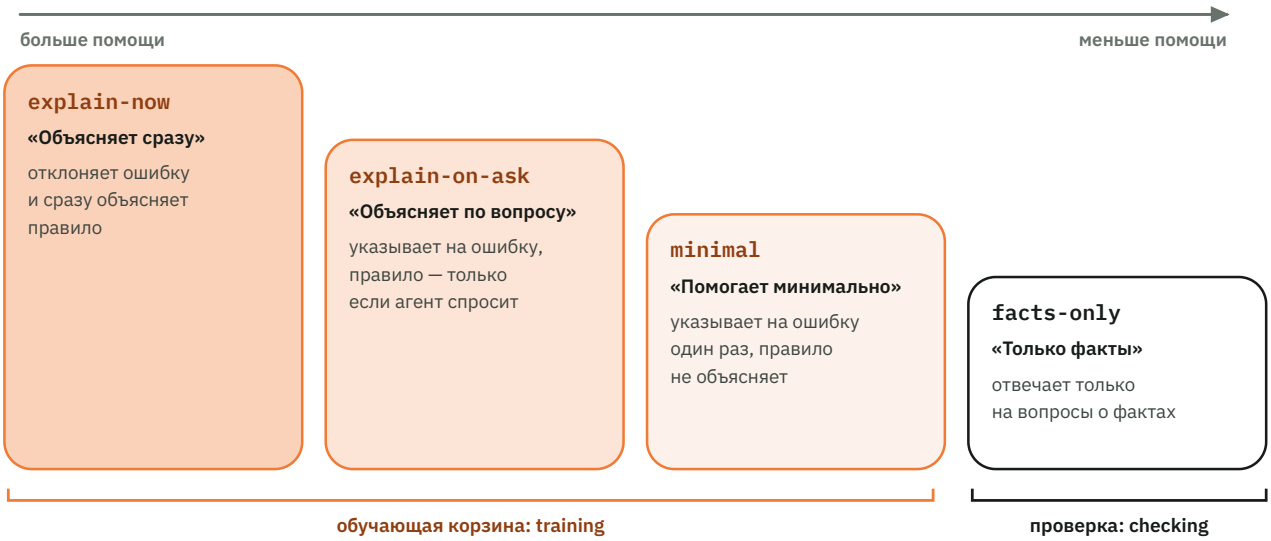


Рис. 10. Лестница помощи. Чем ниже ступень, тем больше агент должен делать сам.

Стиль	Инструкция в демо	Что проверяет
`explain-now` «Объясняет сразу»	Отклоняй действия, которые расходятся с эталоном, и сразу объясняй правило. Если ответ клиенту неполный, скажи, что добавить, и объясни почему.	Усвоит ли агент правило, если его объяснили сразу
`explain-on-ask` «Объясняет по вопросу»	Отклоняй без объяснения и коротко указывай, что не так. Объясняй правило, только если агент спросит.	Научится ли агент сам спрашивать о причине отказа
`minimal` «Помогает минимально»	Помогай минимально: указывай на ошибку один раз, правило не объясняй даже по вопросу.	Как агент справляется без объяснений
`facts-only` «Только факты»	Отвечай только на вопросы о фактах кейса. Не исправляй агента и не объясняй процедуру. Если факта нет в данных, скажи, что таких данных нет.	Экзамен: что агент умеет сам

Утверждённого перечня стилей нет — можно заводить свои. Метка `style` видна оператору, интерфейсу и результатам прогона. Например, стиль «Объясняет после повтора»:

```
operator:
  style: explain-on-repeat
  instruction: >
    Первую ошибку только отклоняй и коротко указывай, что не так.
    Если агент повторяет ту же ошибку, объясни правило.
```

Тестовая модель `fake-gigachat` понимает только четыре стиля из демо, поэтому свой стиль проверяйте на стенде с GigaChat.

4.6 Как написать инструкцию

Хорошая инструкция отвечает на четыре вопроса — и не повторяет то, что уже есть в эталоне:

1

Когда вмешиваться?

На любое расхождение с эталоном, только на запрещённые действия, только в конце хода.

2

Как поправлять?

Отклонить действие, указать на ошибку, подсказать, что добавить в ответ.

3

Объяснять ли правило?

Сразу, по вопросу агента, после повтора или никогда.

4

Что отвечать на вопросы?

Факты кейса, объяснение правила, короткое «решите сами».

Как не надо	Почему	Как надо
«Помогай агенту»	Оператор не знает, насколько помогать	«Отклоняй без объяснения и коротко указывай, что не так»
«Правильный ответ — бонусы ожидают подтверждения партнёром»	Дублирует эталон и устареет при правке кейса	Ответ — в эталоне, в инструкции — стратегия
«Подскажи агенту ответ дословно»	Агент учится на поправках, а не на готовых ответах	«Если ответ неполный, скажи, что добавить, и объясни почему»
«Сначала объясняй сразу, потом только по вопросу»	Две стратегии: результат нельзя приписать ни одной	Два решения
«Сразу объясняй правило» в решении для экзамена	На экзамене оператор в конце хода её не читает	Отдельное решение «Только факты»

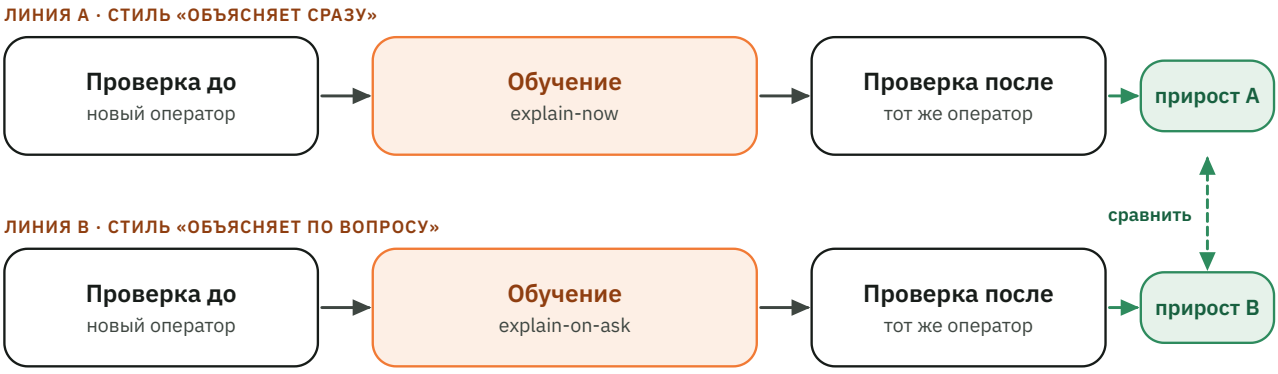
4.7 Поля решения

Поле	Что значит
`case`	Обязательно. Идентификатор кейса
`operator.instruction`	Обязательно. Стратегия помощи — главное, чем решения одного кейса отличаются друг от друга
`title`, `goal`	Название и цель эксперимента: видны в корзине, на странице кейса и в результатах. Оператору цель не передаётся
`operator.style`	Короткая метка стиля; одинаковая у решений одной стратегии в разных кейсах
`operator.seesToolResults`	По умолчанию `true`. `false` — оператор видит вызовы инструментов, но не их ответы: судит только по словам агента
`operator.lessons`	Уроки только для этого сценария
`limits`	Лимиты ходов, времени, запросов оператора и напоминаний, если они нужны стратегии

Идентификатор решения по умолчанию — путь файла без `solutions/` и расширения: `card-hold-amount/explain-now`. По нему на решение ссылается корзина. Перенесли или переименовали файл — поправьте корзины. Лимиты берутся с самого частного уровня: элемент корзины, затем решение, затем настройка прогона, по умолчанию — 20 ходов, 900 секунд, 30 запросов оператора и, на экзамене, 2 напоминания без действий.

4.8 Как собирать решения в эксперименты

- **Занятие — от большей помощи к меньшей.** «Объясняет сразу», потом «по вопросу», потом «минимально», чередуя тематики. Так видно, держится ли выученное без подсказок.
- **Экзамен — только «Только факты».** Один кейс — один элемент экзамена.
- **Сравнить два стиля.** Заведите две обучающие корзины на одних кейсах, каждая со своим стилем, и запустите их на новых операторах. После каждой прогоните одну и ту же проверку на обученном операторе — у каждого стиля своя пара «до и после».



Одни и те же кейсы и проверяющая корзина в обеих линиях; меняется только стиль наставника в обучении.

Рис. 11. Эксперимент «какой стиль учит лучше». Две линии на одних кейсах, у каждой своя проверка до и после обучения. Сравниваются приросты M1 и M2, а не результаты одной проверки.

4.9 Лучшие практики решений

Делайте так

- **Одна стратегия — одно решение**; цель эксперимента записана в `goal`.
- **Инструкция — о том, как помогать**, а не о том, что правильно: правильное оператор видит в эталоне.
- **Коротко и в повелительном наклонении** — одно-три предложения.
- **Одинаковая метка стиля** у одинаковых стратегий во всех кейсах: по ней удобно сравнивать результаты.
- **Отдельное решение для экзамена** — «Только факты».
- **Лимиты в решении** — только когда они нужны стратегии: сценарию «по вопросу» нужно больше ходов, агент должен догадаться спросить.

ЧАСТЬ 5

Корзины — занятие и экзамен

Корзина — программа: какие решения, в каком порядке и учится ли агент. Тип корзины решает, меняются ли скиллы тестового оператора.

5.1 Два типа: занятие и экзамен

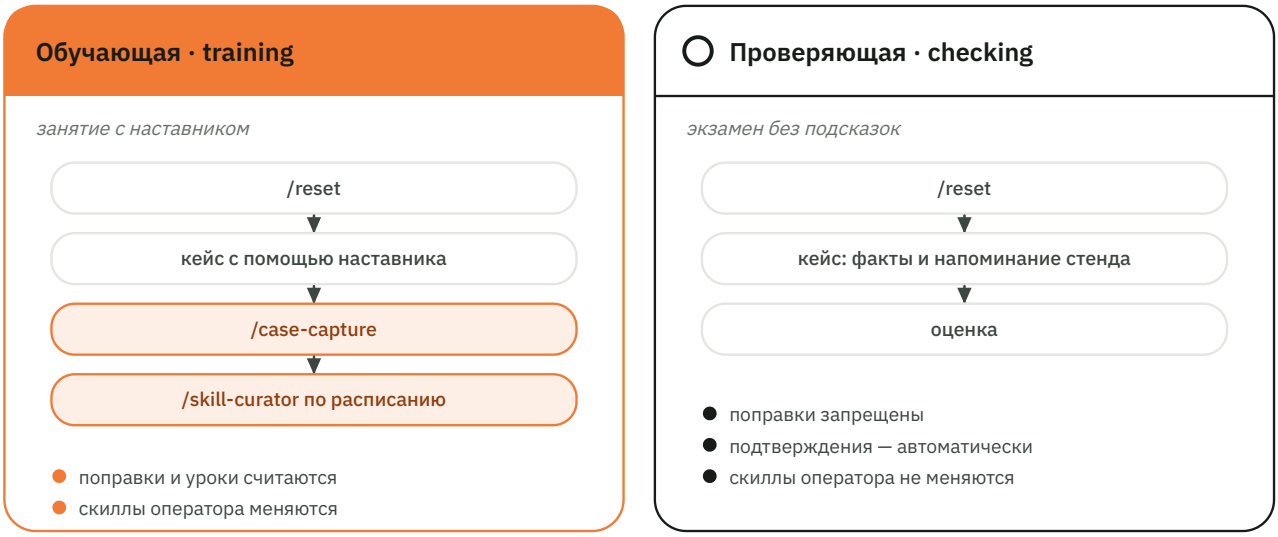


Рис. 12. Обучающая корзина — занятие с наставником, проверяющая — экзамен без подсказок. Перед каждым кейсом в обеих отправляется `/reset`.

	Обучающая · `training`	Проверяющая · `checking`
Зачем	Научить тестового оператора	Измерить, что умеют его скиллы
`/case-capture`	После каждого завершённого кейса, даже решённого неверно	Никогда
`/skill-curator`	По расписанию корзины	Никогда: блок с проходами и шаги — ошибка
Наставник	По инструкции решения	Только факты; решения он не знает
Скиллы оператора	Меняются	Не меняются

Что агент получает на экзамене. После хода агента наставник выбирает одно из трёх. Агент спросил о факте — получает ответ из фактов кейса или «таких данных нет». Агент прямо сказал, что закончил работу, — кейс завершён. Во всех остальных случаях — ждёт указаний, спрашивает, что делать или можно ли закрывать, — агент получает напоминание.

Наставник при этом видит только факты кейса и журнал диалога — ни ситуации, ни эталона, ни инструкции решения: подсказать решение ему нечем.

Напоминание пишет стенд, а не модель

Текст всегда один: «Обращение ведёте вы: действуйте так, как считаете верным. Подсказывать я не буду. Если не хватает данных — спросите. Если считаете работу законченной — так и скажите». Оставил агент без единого вызова инструмента два напоминания подряд — кейс завершается статусом «Исчерпан лимит напоминаний» и оценивается как обычно. Число задаёт лимит `maxReminders`.

5.2 Как устроен файл

```yaml
# baskets/second-line-train.yaml
id: second-line-train
title: Обучение · СберСпасибо и карты
type: training
curator:
  every: 2        # после каждых 2 кейсов
  final: true     # и в конце
items:
  - solution: spasibo-bonus-missing/explain-now
  - solution: card-operation-dispute/explain-now
  - solution: spasibo-bonus-missing/explain-on-ask
  # …
```

```yaml
# baskets/second-line-check.yaml
id: second-line-check
title: Проверка · СберСпасибо и карты
type: checking
checking:
  confirmations: approve
items:
  - solution: spasibo-bonus-missing/facts-only
  - solution: card-operation-dispute/facts-only
  - solution: card-hold-amount/facts-only
```

- **Порядок фиксирован** и одинаков для каждой линии прогона. Решение может повторяться.
- **Корзины лежат только в корне** `baskets/` — файлы в подкаталогах не читаются.
- `id` **корзины попадает в идентификатор тестового оператора**, поэтому он короткий, латиницей, без `/`.
- **Лимиты для одной позиции** задаются прямо в элементе: `limits: { maxTurns: 12 }`.

5.3 Когда запускать `/skill-curator`

Curator переносит уроки из сохранённых случаев в выученные скиллы. ClawGym ведёт счётчик — сколько кейсов завершено после прошлого прохода — и запускает curator по расписанию корзины.



every: 2 — проход после каждых двух завершённых кейсов, счётчик обнуляется;
final: true — после седьмого кейса счётчик не нулевой, поэтому финальный проход.

Рис. 13. Расписание в демо-корзине обучения из семи решений. План строится заранее и виден на странице «Корзины» и в `preview`.

Правило	Как работает
`every: N`	Счётчик дошёл до N — проход, счётчик в ноль
`final: true`	Корзина кончилась, а счётчик не нулевой — финальный проход
`- curator: true`	Явный шаг в списке: проход на этом месте, всегда, и без блока `curator`
Технический сбой кейса	`/case-capture` не отправляется, счётчик не растёт — проходы сдвигаются

Как часто запускать curator

Свежий урок агент видит сразу, в начале следующего обращения, поэтому curator не обязан быть частым: его задача — переносить уроки в скиллы. Оркестратору рекомендован проход после пяти и более сохранённых кейсов с сильным или средним сигналом. Агент показывает не больше десяти свежих уроков, поэтому:

- `every: 5` — близко к рабочей политике; в коротких корзинах меньше, как `every: 2` в демо;
- между проходами — не больше десяти поправок;
- `final: true`, если после занятия идёт экзамен: уроки последних кейсов попадут в скиллы.

5.4 Экзамен «до и после»

- **Заморозьте проверяющую корзину на всю серию.** Не меняйте её состав и порядок, кейсы и решения между проверками «до» и «после». Любая правка, даже опечатка в критерии, делает прогоны несопоставимыми — тогда серию начинают заново.
- **Кейсы сравниваются по позиции:** на позиции должно стоять то же решение.
- **Обобщение, а не память.** Кроме кейсов из занятия, добавьте в экзамен кейсы тех же тематик с другими данными.
- **Разные исходы до обучения.** Нужны кейсы, где агент ошибается, — на них видно обучение, и кейсы, где он прав, — на них видны регрессии.
- **Повторения.** При меньше чем трёх повторениях разницу нельзя считать статистически значимой.
- **Один лимит напоминаний на всю серию:** с разным `maxReminders` прогоны несопоставимы.
- **Большой экзамен — это часы:** он идёт последовательно на одном обученном операторе. Ставьте его на ночь — поле `startAt` в настройке прогона.

5.5 Лучшие практики корзин

Делайте так

- **Пары с понятными именами:** `<область>-train` и `<область>-check`, названия «Обучение · …» и «Проверка · …».
- **Чередуйте тематики и стратегии** в обучении: между повторами одного кейса агент решает другие обращения — так видно, держится ли урок.
- **Одно правило — в нескольких кейсах** с общим уроком: повторные поправки покажут, усвоено ли оно.
- **Явный шаг curator не ставьте сразу за периодическим проходом** — будет два прохода подряд, второму нечего разбирать.
- **Сначала маленькая корзина:** новый кейс отлаживайте в корзине из одного-двух элементов.
- **Смотрите план** на странице «Корзины» или в `preview` перед каждым новым прогоном.

ЧАСТЬ 6

Шпаргалка

Путь от идеи до прогона, чек-листы и частые сообщения проверки файлов.

6.1 От идеи до прогона

1. **Проверьте инструменты** в `contracts/tools.json`: какие нужны агенту, какие требуют подтверждения, какие закрывают кейс.
2. **Напишите кейс** — начните с копии похожего из `dev/config/cases/`.
3. **Проверьте файлы:** `npm run validate` или «Перечитать файлы» на странице «Кейсы и решения». Откройте стартовое сообщение.
4. **Пройдите кейс вручную.** Если для вызова нет заглушки, ответьте вместо инструмента — ClawGym покажет готовый YAML-фрагмент для кейса.
5. **Напишите решения:** хотя бы одно обучающее и «Только факты».
6. **Соберите корзины:** обучающую и проверяющую.
7. **Заведите настройки прогона** по схеме «проверка → обучение → та же проверка».
8. **Посмотрите план** — «Запуск прогона» или `node dist/src/cli.js preview <настройка>` — и запускайте.

6.2 Чек-листы

Кейс

- [ ] `id` = имя файла, латиница
- [ ] `topic` совпадает с `categoryName`
- [ ] Факты — данные, без правил; всё, что можно сообщить агенту, — в них
- [ ] Заглушки старта — без `when`
- [ ] Есть заглушки типичной ошибки
- [ ] Значения с `*` и `+` — в кавычках
- [ ] 2–4 атомарных критерия
- [ ] `finalClass`, `tools`, `finalText`
- [ ] Пройден вручную

Урок

- [ ] Одно правило с понятным действием
- [ ] Без сумм и дат конкретного кейса
- [ ] Стабильный `id` вида `L-тема-номер`
- [ ] Тот же `id` для того же правила в других кейсах
- [ ] Связан с действием или критерием
- [ ] Не меняется внутри серии

Решение

- [ ] `case` указывает на существующий кейс
- [ ] `title` и `goal` объясняют эксперимент
- [ ] Инструкция — стратегия, а не ответ
- [ ] Одна стратегия в решении
- [ ] Для экзамена — «Только факты»
- [ ] Лимиты — только если нужны стратегии

Корзина

- [ ] Файл в корне `baskets/`, короткий `id`
- [ ] Тип выбран осознанно
- [ ] Все решения существуют
- [ ] Обучение: расписание curator и проход в конце
- [ ] Экзамен: «Только факты», `approve`
- [ ] Экзамен заморожен на всю серию

6.3 Частые сообщения проверки файлов

Сообщение	Что делать
Нет эталонного класса финального действия: кейс не войдёт в M1	Задайте `reference.finalClass`
Нет эталонного набора инструментов: F1 и M2 не посчитаются	Задайте `reference.tools`
Нет критериев ответа: LLM-оценке нечего проверять	Добавьте `reference.criteria`
Урок «...» не описан в reference.lessons	Добавьте урок в каталог или исправьте ссылку
Значение «...» YAML читает как ссылку на якорь	Возьмите значение в кавычки
В result нужно ровно одно из json, text, file	Оставьте один вид ответа
Решение «...» не найдено в solutions/	Исправьте ссылку в корзине — после переноса файла решения идентификатор меняется
В проверяющей корзине /skill-curator не запускается	Уберите блок `curator` или смените тип корзины
Подключение не поддерживает /skill-curator, а корзина его запускает	Задайте команду в подключении агента

6.4 Где читать подробнее

Документ	Что там
`docs/guide/README.md`	Обзор, путь от идеи до прогона, проверка файлов, словарь
`docs/guide/cases.md`	Все поля кейса, условия заглушек, шаблоны, стартовое сообщение, оценка
`docs/guide/solutions.md`, `baskets.md`	Решения и корзины: поля, лимиты, политика curator, примеры
`docs/guide/cheatsheet.md`	Все поля на одной странице и все сообщения проверки файлов
`docs/FORMATS.md`, `docs/CONTRACTS.md`	Справочник форматов и правила, принятые реализацией